SYNAPSE Database Schema Design

Technology Intelligence Platform
AI-Powered Architecture

March 7, 2026

Contents

0.1 repositories Table . . . 2
0.2 research_papers Table . . . 3
0.3 videos Table . . . 4

1 User Interaction Tables **5**
1.1 user_bookmarks Table . . . 5
1.2 collections Table . . . 6
1.3 collection_items Table . . . 7

2 Automation and Task Management **8**
2.1 automation_workflows Table . . . 8
2.2 workflow_runs Table . . . 9
2.3 agent_tasks Table . . . 10
2.4 generated_documents Table . . . 11

3 Analytics and Notifications **12**
3.1 technology_trends Table . . . 12
3.2 notifications Table . . . 13
3.3 user_activity Table . . . 14

4 Executive Summary **15**

5 Indexing Strategy **16**
5.1 Index Categories . . . 16
5.1.1 Primary Key Indexes (B-Tree) . . . 16
5.1.2 Foreign Key Indexes (B-Tree) . . . 16
5.1.3 Unique Constraints (B-Tree) . . . 16
5.1.4 Array/JSONB Indexes (GIN - Generalized Inverted Index) . . . 17
5.1.5 Timestamp Indexes (B-Tree) . . . 18
5.1.6 Composite Indexes . . . 18
5.2 Indexing Best Practices . . . 18

6 System Architecture Overview **19**
6.1 Technology Stack . . . 19
6.2 Design Principles . . . 19

7 Core Tables **20**
7.1 users Table . . . 20
7.1.1 Table Definition . . . 20
7.1.2 SQL Definition . . . 20

7.2 sources Table . . . 22
7.3 articles Table . . . 23

8 Vector Database Architecture **24**
8.1 Overview . . . 24
8.2 Vector Collections . . . 24
8.2.1 Articles Embeddings . . . 24
8.2.2 Pinecone Collections . . . 24
8.3 Embedding Metadata . . . 24

9 Redis Caching Strategy **26**
9.1 Key Patterns and TTLs . . . 26
9.1.1 User Session Cache . . . 26
9.1.2 Content Cache . . . 26
9.1.3 Aggregate Caches . . . 26
9.1.4 Task Queue Cache . . . 27
9.2 Cache Invalidation Strategy . . . 27

10 Entity Relationship Overview **28**
10.1 Core Relationships . . . 28
10.2 Polymorphic Relationships . . . 28

11 Database Performance Considerations **29**
11.1 Partitioning Strategy . . . 29
11.1.1 Time-Based Partitioning . . . 29
11.1.2 Range-Based Partitioning . . . 29
11.2 Query Optimization Tips . . . 29
11.3 Monitoring and Maintenance . . . 30
11.3.1 Key Metrics to Monitor . . . 30
11.3.2 Maintenance Operations . . . 30

12 Data Migration and Backup Strategy **31**
12.1 Backup Approach . . . 31
12.2 Migration Path . . . 31

13 Security and Compliance **32**
13.1 Access Control . . . 32
13.2 Sensitive Data Handling . . . 32

14 Conclusion **33**

0.1 repositories Table

GitHub repositories tracked for trending analysis and intelligence gathering.

Table 1: repositories Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique repository identifier
github_id	BIGINT	UNIQUE, NOT NULL	GitHub repository ID
name	VARCHAR(500)	NOT NULL	Repository name
full_name	VARCHAR(500)	NOT NULL	Full repository path
description	TEXT	NULL	Repository description
url	VARCHAR(1000)	NOT NULL	GitHub repository URL
clone_url	VARCHAR(1000)	NULL	Git clone URL
stars	INTEGER	DEFAULT 0	Star count
forks	INTEGER	DEFAULT 0	Fork count
watchers	INTEGER	DEFAULT 0	Watcher count
language	VARCHAR(100)	NULL	Primary programming language
topics	TEXT[]	NULL	Repository topics/tags
owner	VARCHAR(200)	NOT NULL	Repository owner username
is_trending	BOOLEAN	DEFAULT FALSE	Trending indicator
scraped_at	TIMESTAMP	NOT NULL	Last data collection
created_at	TIMESTAMP	NOT NULL	Repository creation time
readme_summary	TEXT	NULL	Extracted README summary
embedding_id	VARCHAR(200)	NULL	Vector DB reference

```
CREATE TABLE repositories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    github_id BIGINT UNIQUE NOT NULL,
    name VARCHAR(500) NOT NULL,
    full_name VARCHAR(500) NOT NULL,
    description TEXT,
    url VARCHAR(1000) NOT NULL,
    clone_url VARCHAR(1000),
    stars INTEGER DEFAULT 0,
    forks INTEGER DEFAULT 0,
    watchers INTEGER DEFAULT 0,
    language VARCHAR(100),
    topics TEXT[],
    owner VARCHAR(200) NOT NULL,
    is_trending BOOLEAN DEFAULT FALSE,
    scraped_at TIMESTAMP NOT NULL DEFAULT NOW(),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    readme_summary TEXT,
    embedding_id VARCHAR(200)
);
```

0.2 research_papers Table

Academic research papers and preprints from arXiv and similar sources.

Table 2: research_papers Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique paper identifier
arxiv_id	VARCHAR(50)	UNIQUE, NOT NULL	arXiv paper ID
title	TEXT	NOT NULL	Paper title
abstract	TEXT	NULL	Paper abstract
summary	TEXT	NULL	AI-generated summary
authors	TEXT[]	NOT NULL	Array of author names
categories	TEXT[]	NULL	arXiv category tags
published_date	DATE	NULL	Publication date
url	VARCHAR(1000)	NOT NULL	Paper URL
pdf_url	VARCHAR(1000)	NULL	PDF download URL
citation_count	INTEGER	DEFAULT 0	Citation count
difficulty_level	VARCHAR(50)	NULL	beginner, intermediate, advanced
key_contributions	TEXT	NULL	Paper key contributions
applications	TEXT	NULL	Practical applications
embedding_id	VARCHAR(200)	NULL	Vector DB reference
fetched_at	TIMESTAMP	NOT NULL	Data retrieval timestamp

```
CREATE TABLE research_papers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    arxiv_id VARCHAR(50) UNIQUE NOT NULL,
    title TEXT NOT NULL,
    abstract TEXT,
    summary TEXT,
    authors TEXT[] NOT NULL,
    categories TEXT[],
    published_date DATE,
    url VARCHAR(1000) NOT NULL,
    pdf_url VARCHAR(1000),
    citation_count INTEGER DEFAULT 0,
    difficulty_level VARCHAR(50) CHECK (difficulty_level IN
        ('beginner', 'intermediate', 'advanced')),
    key_contributions TEXT,
    applications TEXT,
    embedding_id VARCHAR(200),
    fetched_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

0.3 videos Table

Video content from YouTube and other video platforms.

Table 3: videos Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique video identifier
youtube_id	VARCHAR(50)	UNIQUE, NOT NULL	YouTube video ID
title	TEXT	NOT NULL	Video title
description	TEXT	NULL	Video description
summary	TEXT	NULL	AI-generated summary
channel_name	VARCHAR(300)	NOT NULL	Channel name
channel_id	VARCHAR(300)	NOT NULL	YouTube channel ID
url	VARCHAR(1000)	NOT NULL	Video URL
thumbnail_url	VARCHAR(1000)	NULL	Thumbnail image URL
duration_seconds	INTEGER	NULL	Video duration in seconds
view_count	INTEGER	DEFAULT 0	View count
like_count	INTEGER	DEFAULT 0	Like count
published_at	TIMESTAMP	NULL	Upload date
transcript	TEXT	NULL	Video transcript
topics	TEXT[]	NULL	Video topics/tags
embedding_id	VARCHAR(200)	NULL	Vector DB reference

```
CREATE TABLE videos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    youtube_id VARCHAR(50) UNIQUE NOT NULL,
    title TEXT NOT NULL,
    description TEXT,
    summary TEXT,
    channel_name VARCHAR(300) NOT NULL,
    channel_id VARCHAR(300) NOT NULL,
    url VARCHAR(1000) NOT NULL,
    thumbnail_url VARCHAR(1000),
    duration_seconds INTEGER,
    view_count INTEGER DEFAULT 0,
    like_count INTEGER DEFAULT 0,
    published_at TIMESTAMP,
    transcript TEXT,
    topics TEXT[],
    embedding_id VARCHAR(200)
);
```

Chapter 1

User Interaction Tables

1.1 user_bookmarks Table

Bookmarks and saved content by users across all content types.

Table 1.1: user_bookmarks Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique bookmark identifier
user_id	UUID	FK, NOT NULL	Reference to users table
content_type	VARCHAR(50)	NOT NULL	article, repository, research_paper, video
content_id	UUID	NOT NULL	Reference to content
notes	TEXT	NULL	User personal notes
tags	TEXT[]	NULL	User-assigned tags
created_at	TIMESTAMP	NOT NULL	Bookmark creation time

```
CREATE TABLE user_bookmarks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    content_type VARCHAR(50) NOT NULL CHECK (content_type IN
        ('article', 'repository', 'research_paper', 'video')),
    content_id UUID NOT NULL,
    notes TEXT,
    tags TEXT[],
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

1.2 collections Table

User-curated collections of content.

Table 1.2: collections Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique collection identifier
user_id	UUID	FK, NOT NULL	Reference to users table
name	VARCHAR(500)	NOT NULL	Collection name
description	VARCHAR(500)	NULL	Collection description
is_public	BOOLEAN	DEFAULT FALSE	Public visibility flag
created_at	TIMESTAMP	NOT NULL	Creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

```
CREATE TABLE collections (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    name VARCHAR(500) NOT NULL,
    description VARCHAR(500),
    is_public BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

1.3 collection_items Table

Individual items within collections.

Table 1.3: collection_items Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique collection item identifier
collection_id	UUID	FK, NOT NULL	Reference to collections table
content_type	VARCHAR(50)	NOT NULL	article, repository, research_paper, video
content_id	UUID	NOT NULL	Reference to content
added_at	TIMESTAMP	NOT NULL	Item addition timestamp

```
CREATE TABLE collection_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    collection_id UUID NOT NULL REFERENCES collections(id),
    content_type VARCHAR(50) NOT NULL CHECK (content_type IN
        ('article', 'repository', 'research_paper', 'video')),
    content_id UUID NOT NULL,
    added_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

Chapter 2

Automation and Task Management

2.1 automation_workflows Table

User-defined automation workflows and scheduled tasks.

Table 2.1: automation_workflows Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique workflow identifier
user_id	UUID	FK, NOT NULL	Reference to users table
name	TEXT	NOT NULL	Workflow name
description	TEXT	NULL	Workflow description
trigger_type	VARCHAR(50)	NOT NULL	schedule, event, manual
cron_expression	VARCHAR(100)	NULL	CRON schedule (if scheduled)
actions	JSONB	NOT NULL	Workflow action definitions
is_active	BOOLEAN	DEFAULT TRUE	Activation status
last_run_at	TIMESTAMP	NULL	Last execution timestamp
next_run_at	TIMESTAMP	NULL	Next scheduled execution
run_count	INTEGER	DEFAULT 0	Total execution count
created_at	TIMESTAMP	NOT NULL	Creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last modification timestamp

```
CREATE TABLE automation_workflows (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    name TEXT NOT NULL,
    description TEXT,
    trigger_type VARCHAR(50) NOT NULL CHECK (trigger_type IN
        ('schedule', 'event', 'manual')),
    cron_expression VARCHAR(100),
    actions JSONB NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    last_run_at TIMESTAMP,
    next_run_at TIMESTAMP,
    run_count INTEGER DEFAULT 0,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

2.2 workflow_runs Table

Execution history of automation workflows.

Table 2.2: workflow_runs Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique run identifier
workflow_id	UUID	FK, NOT NULL	Reference to automation_workflows
status	VARCHAR(50)	NOT NULL	pending, running, success, failed
started_at	TIMESTAMP	NOT NULL	Execution start time
completed_at	TIMESTAMP	NULL	Execution completion time
result	JSONB	NULL	Execution result data
error_message	TEXT	NULL	Error message if failed

```
CREATE TABLE workflow_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID NOT NULL
        REFERENCES automation_workflows(id),
    status VARCHAR(50) NOT NULL CHECK (status IN
        ('pending', 'running', 'success', 'failed')),
    started_at TIMESTAMP NOT NULL DEFAULT NOW(),
    completed_at TIMESTAMP,
    result JSONB,
    error_message TEXT
);
```

2.3 agent_tasks Table

AI agent task requests and execution tracking.

Table 2.3: agent_tasks Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique task identifier
user_id	UUID	FK, NOT NULL	Reference to users table
task_type	VARCHAR(200)	NOT NULL	Task classification
prompt	TEXT	NOT NULL	User prompt/instruction
status	VARCHAR(50)	NOT NULL	pending, processing, completed, failed
result	JSONB	NULL	Task execution result
created_at	TIMESTAMP	NOT NULL	Task creation time
completed_at	TIMESTAMP	NULL	Task completion time
celery_task_id	VARCHAR(200)	NULL	Celery task identifier

```
CREATE TABLE agent_tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    task_type VARCHAR(200) NOT NULL,
    prompt TEXT NOT NULL,
    status VARCHAR(50) NOT NULL CHECK (status IN
        ('pending', 'processing', 'completed', 'failed')),
    result JSONB,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    completed_at TIMESTAMP,
    celery_task_id VARCHAR(200)
);
```

2.4 generated_documents Table

AI-generated reports and documents created by the system.

Table 2.4: generated_documents Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique document identifier
user_id	UUID	FK, NOT NULL	Reference to users table
title	VARCHAR(500)	NOT NULL	Document title
doc_type	VARCHAR(50)	NOT NULL	pdf, ppt, word, markdown
file_path	VARCHAR(1000)	NOT NULL	Local file storage path
cloud_url	VARCHAR(1000)	NULL	Cloud storage URL
file_size_bytes	BIGINT	NULL	Document file size
agent_prompt	TEXT	NULL	Original generation prompt
created_at	TIMESTAMP	NOT NULL	Generation timestamp
metadata	JSONB	DEFAULT '{}'	Additional document metadata

```
CREATE TABLE generated_documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    title VARCHAR(500) NOT NULL,
    doc_type VARCHAR(50) NOT NULL CHECK (doc_type IN
        ('pdf', 'ppt', 'word', 'markdown')),
    file_path VARCHAR(1000) NOT NULL,
    cloud_url VARCHAR(1000),
    file_size_bytes BIGINT,
    agent_prompt TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    metadata JSONB DEFAULT '{}'
);
```

Chapter 3

Analytics and Notifications

3.1 technology_trends Table

Aggregated technology trending metrics across platform.

Table 3.1: technology_trends Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique trend identifier
technology_name	VARCHAR(200)	NOT NULL	Technology or framework name
mention_count	INTEGER	DEFAULT 0	Total mentions across sources
trend_score	FLOAT	NULL	Calculated trend score (0-100)
category	VARCHAR(100)	NULL	Technology category
date	DATE	NOT NULL	Trend date
sources	TEXT[]	NULL	Array of contributing sources

```
CREATE TABLE technology_trends (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    technology_name VARCHAR(200) NOT NULL,
    mention_count INTEGER DEFAULT 0,
    trend_score FLOAT,
    category VARCHAR(100),
    date DATE NOT NULL,
    sources TEXT[]
);
```

3.2 notifications Table

User notifications for activities and alerts.

Table 3.2: notifications Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique notification identifier
user_id	UUID	FK, NOT NULL	Reference to users table
title	TEXT	NOT NULL	Notification title
message	TEXT	NOT NULL	Notification message
notif_type	VARCHAR(100)	NOT NULL	alert, info, success, warning
is_read	BOOLEAN	DEFAULT FALSE	Read status
created_at	TIMESTAMP	NOT NULL	Creation timestamp
metadata	JSONB	DEFAULT '{}'	Additional metadata

```
CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    title TEXT NOT NULL,
    message TEXT NOT NULL,
    notif_type VARCHAR(100) NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    metadata JSONB DEFAULT '{}'
);
```

3.3 user_activity Table

User interaction tracking and analytics.

Table 3.3: user_activity Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique activity record identifier
user_id	UUID	FK, NOT NULL	Reference to users table
action_type	VARCHAR(100)	NOT NULL	viewed, liked, bookmarked, shared
content_type	VARCHAR(100)	NULL	article, repository, etc.
content_id	VARCHAR(100)	NULL	ID of interacted content
created_at	TIMESTAMP	NOT NULL	Activity timestamp
metadata	JSONB	DEFAULT '{}'	Additional context

```
CREATE TABLE user_activity (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    action_type VARCHAR(100) NOT NULL,
    content_type VARCHAR(100),
    content_id VARCHAR(100),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    metadata JSONB DEFAULT '{}'
);
```

Chapter 4

Executive Summary

This document provides a comprehensive database schema design for SYNAPSE, a FAANG-style AI-powered technology intelligence platform. The platform aggregates and analyzes technology news, open-source repositories, academic research, and video content to provide actionable intelligence to users.

The design follows PostgreSQL best practices with proper indexing strategies, support for vector embeddings, caching patterns, and scalability considerations. The schema accommodates multiple data sources, user management, content organization, workflow automation, and real-time notifications.

Chapter 5

Indexing Strategy

5.1 Index Categories

The SYNAPSE database employs a multi-tier indexing strategy to optimize query performance across different access patterns:

5.1.1 Primary Key Indexes (B-Tree)

Automatically created for all UUID primary keys. These ensure fast lookups and join operations.

```
-- Example: Automatic index on users table
CREATE UNIQUE INDEX idx_users_id ON users(id);
```

5.1.2 Foreign Key Indexes (B-Tree)

Critical for join performance when referencing related tables.

```
-- Foreign key relationships
CREATE INDEX idx_articles_source_id ON articles(source_id);
CREATE INDEX idx_user_bookmarks_user_id ON user_bookmarks(user_id);
CREATE INDEX idx_collections_user_id ON collections(user_id);
CREATE INDEX idx_automation_workflows_user_id
    ON automation_workflows(user_id);
CREATE INDEX idx_workflow_runs_workflow_id
    ON workflow_runs(workflow_id);
CREATE INDEX idx_agent_tasks_user_id ON agent_tasks(user_id);
CREATE INDEX idx_generated_documents_user_id
    ON generated_documents(user_id);
CREATE INDEX idx_notifications_user_id ON notifications(user_id);
CREATE INDEX idx_user_activity_user_id ON user_activity(user_id);
```

5.1.3 Unique Constraints (B-Tree)

Enforce data uniqueness and serve as implicit indexes.

```
-- Unique constraint indexes
CREATE UNIQUE INDEX idx_users_username ON users(username);
```

```
CREATE UNIQUE INDEX idx_users_email ON users(email);
CREATE UNIQUE INDEX idx_articles_url ON articles(url);
CREATE UNIQUE INDEX idx_repositories_github_id
    ON repositories(github_id);
CREATE UNIQUE INDEX idx_research_papers_arxiv_id
    ON research_papers(arxiv_id);
CREATE UNIQUE INDEX idx_videos_youtube_id ON videos(youtube_id);
```

5.1.4 Array/JSONB Indexes (GIN - Generalized Inverted Index)

Optimized for searches within array and JSONB columns.

```
-- GIN indexes for array and JSONB columns
CREATE INDEX idx_articles_tags
    ON articles USING GIN(tags);
CREATE INDEX idx_articles_keywords
    ON articles USING GIN(keywords);
CREATE INDEX idx_repositories_topics
    ON repositories USING GIN(topics);
CREATE INDEX idx_research_papers_categories
    ON research_papers USING GIN(categories);
CREATE INDEX idx_research_papers_authors
    ON research_papers USING GIN(authors);
CREATE INDEX idx_videos_topics
    ON videos USING GIN(topics);
CREATE INDEX idx_user_bookmarks_tags
    ON user_bookmarks USING GIN(tags);

-- JSONB column indexes
CREATE INDEX idx_users_preferences
    ON users USING GIN(preferences);
CREATE INDEX idx_sources_config
    ON sources USING GIN(config);
CREATE INDEX idx_articles_metadata
    ON articles USING GIN(metadata);
CREATE INDEX idx_automation_workflows_actions
    ON automation_workflows USING GIN(actions);
CREATE INDEX idx_workflow_runs_result
    ON workflow_runs USING GIN(result);
CREATE INDEX idx_agent_tasks_result
    ON agent_tasks USING GIN(result);
CREATE INDEX idx_generated_documents_metadata
    ON generated_documents USING GIN(metadata);
CREATE INDEX idx_notifications_metadata
    ON notifications USING GIN(metadata);
CREATE INDEX idx_user_activity_metadata
    ON user_activity USING GIN(metadata);
```

5.1.5 Timestamp Indexes (B-Tree)

Essential for time-range queries and temporal filtering.

```
-- Timestamp indexes for range queries
CREATE INDEX idx_articles_published_at ON articles(published_at);
CREATE INDEX idx_articles_scraped_at ON articles(scraped_at);
CREATE INDEX idx_repositories_scraped_at ON repositories(scraped_at);
CREATE INDEX idx_workflow_runs_started_at
    ON workflow_runs(started_at);
CREATE INDEX idx_workflow_runs_completed_at
    ON workflow_runs(completed_at);
CREATE INDEX idx_agent_tasks_created_at ON agent_tasks(created_at);
CREATE INDEX idx_notifications_created_at
    ON notifications(created_at);
CREATE INDEX idx_user_activity_created_at
    ON user_activity(created_at);
CREATE INDEX idx_user_bookmarks_created_at
    ON user_bookmarks(created_at);
```

5.1.6 Composite Indexes

Optimized indexes combining multiple columns for specific query patterns.

```
-- Composite indexes for common query patterns
CREATE INDEX idx_articles_source_published
    ON articles(source_id, published_at DESC);
CREATE INDEX idx_user_bookmarks_user_content
    ON user_bookmarks(user_id, content_type);
CREATE INDEX idx_collection_items_collection_content
    ON collection_items(collection_id, content_type);
CREATE INDEX idx_technology_trends_date_score
    ON technology_trends(date DESC, trend_score DESC);
CREATE INDEX idx_notifications_user_read
    ON notifications(user_id, is_read);
CREATE INDEX idx_user_activity_user_type
    ON user_activity(user_id, action_type);
```

5.2 Indexing Best Practices

- **B-Tree Indexes**: Used for equality and range comparisons (default)
- **GIN Indexes**: Used for JSONB and array containment queries
- **GIST Indexes**: Consider for geometric or full-text search extensions
- **Index Maintenance**: Regular ANALYZE and VACUUM operations required
- **Monitoring**: Track index usage via `pg_stat_user_indexes`
- **Partial Indexes**: Can index only active records to save space

Chapter 6

System Architecture Overview

6.1 Technology Stack

- **Primary Database**: PostgreSQL 14+
- **Vector Database**: Pinecone or pgvector extension
- **Caching Layer**: Redis
- **Task Queue**: Celery with Redis backend
- **File Storage**: Cloud storage (S3-compatible)

6.2 Design Principles

1. **Normalization**: Database follows 3NF for data integrity
2. **Scalability**: Designed for horizontal scaling with read replicas
3. **Performance**: Strategic indexing for query optimization
4. **Flexibility**: JSONB columns for semi-structured data
5. **Auditability**: Timestamp tracking for all entities
6. **Security**: Role-based access control via user roles

Chapter 7

Core Tables

7.1 users Table

User management and authentication for the SYNAPSE platform.

7.1.1 Table Definition

Table 7.1: users Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique user identifier
username	VARCHAR(150)	UNIQUE, NOT NULL	User login name
email	VARCHAR(255)	UNIQUE, NOT NULL	User email address
password_hash	VARCHAR(255)	NULL	Bcrypt/Argon2 password hash
first_name	VARCHAR(100)	NULL	User first name
last_name	VARCHAR(100)	NULL	User last name
role	ENUM	NOT NULL	admin, user, premium
is_active	BOOLEAN	DEFAULT TRUE	Account status
created_at	TIMESTAMP	NOT NULL	Account creation time
updated_at	TIMESTAMP	NOT NULL	Last profile update
last_login	TIMESTAMP	NULL	Last authentication timestamp
preferences	JSONB	DEFAULT '{}'	User settings and preferences
avatar_url	VARCHAR(500)	NULL	Profile image URL

7.1.2 SQL Definition

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(150) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255),
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    role VARCHAR(50) NOT NULL CHECK (role IN
        ('admin', 'user', 'premium')),
```

```
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
    last_login TIMESTAMP,
    preferences JSONB DEFAULT '{}',
    avatar_url VARCHAR(500)
);
```

7.2 sources Table

Content source configuration for automated data collection.

Table 7.2: sources Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique source identifier
name	VARCHAR(200)	NOT NULL	Source display name
url	VARCHAR(500)	UNIQUE, NULL	Source base URL
source_type	VARCHAR(50)	NOT NULL	news, github, arxiv, youtube, blog
is_active	BOOLEAN	NULL	Scraping enabled status
scrape_interval_minutes	INTEGER	NULL	Interval between scrapes
last_scraped_at	TIMESTAMP	NULL	Last successful scrape
config	JSONB	DEFAULT '{}'	Source-specific configuration
created_at	TIMESTAMP	NOT NULL	Creation timestamp

```
CREATE TABLE sources (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(200) NOT NULL,
    url VARCHAR(500) UNIQUE,
    source_type VARCHAR(50) NOT NULL CHECK (source_type IN
        ('news', 'github', 'arxiv', 'youtube', 'blog')),
    is_active BOOLEAN DEFAULT TRUE,
    scrape_interval_minutes INTEGER,
    last_scraped_at TIMESTAMP,
    config JSONB DEFAULT '{}',
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

7.3 articles Table

News articles and blog posts aggregated from various sources.

Table 7.3: articles Table Schema

Column Name	Data Type	Constraints	Description
id	UUID	PK	Unique article identifier
title	TEXT	NOT NULL	Article headline
content	TEXT	NULL	Full article body
summary	TEXT	NULL	Generated summary
url	VARCHAR(1000)	UNIQUE, NULL	Original article URL
source_id	UUID	FK, NOT NULL	Reference to sources table
author	VARCHAR(300)	NULL	Article author name
published_at	TIMESTAMP	NULL	Publication date
scraped_at	TIMESTAMP	NOT NULL	Data collection timestamp
topic	VARCHAR(100)	NULL	Article topic classification
tags	TEXT[]	NULL	Array of tags
keywords	TEXT[]	NULL	Array of extracted keywords
sentiment_score	FLOAT	NULL	-1.0 to 1.0 sentiment analysis
trending_score	FLOAT	NULL	Trending metric (0-100)
view_count	INTEGER	DEFAULT 0	Platform view count
embedding_id	VARCHAR(200)	NULL	Vector DB reference
metadata	JSONB	DEFAULT '{}'	Additional metadata

```
CREATE TABLE articles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title TEXT NOT NULL,
    content TEXT,
    summary TEXT,
    url VARCHAR(1000) UNIQUE,
    source_id UUID NOT NULL REFERENCES sources(id),
    author VARCHAR(300),
    published_at TIMESTAMP,
    scraped_at TIMESTAMP NOT NULL DEFAULT NOW(),
    topic VARCHAR(100),
    tags TEXT[],
    keywords TEXT[],
    sentiment_score FLOAT CHECK (sentiment_score >= -1.0
        AND sentiment_score <= 1.0),
    trending_score FLOAT,
    view_count INTEGER DEFAULT 0,
    embedding_id VARCHAR(200),
    metadata JSONB DEFAULT '{}'
);
```

Chapter 8

Vector Database Architecture

8.1 Overview

SYNAPSE leverages a dedicated vector database (Pinecone or pgvector) to store embeddings for semantic search and similarity matching across all content types.

8.2 Vector Collections

8.2.1 Articles Embeddings

```
-- If using pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Add vector column to articles table
ALTER TABLE articles ADD COLUMN embedding vector(1536);

-- Create index for similarity search
CREATE INDEX ON articles USING ivfflat
    (embedding vector_cosine_ops) WITH (lists = 100);
```

8.2.2 Pinecone Collections

For larger deployments, Pinecone provides managed vector storage:

- **articles-embeddings**: 1536-dim embeddings from text-embedding-3-small
- **repositories-embeddings**: Repository README and description embeddings
- **research-papers-embeddings**: Academic paper abstract embeddings
- **videos-embeddings**: Video transcript and description embeddings

8.3 Embedding Metadata

Vector database records maintain the following metadata for filtering:

```
{
    "content_id": "uuid-string",
    "content_type": "article|repository|paper|video",
    "source_id": "uuid-string",
    "published_date": "ISO-8601",
    "trending_score": 85.5,
    "language": "en",
    "tags": ["ai", "machine-learning"]
}
```

Chapter 9

Redis Caching Strategy

9.1 Key Patterns and TTLs

Redis provides high-speed caching for frequently accessed data:

9.1.1 User Session Cache

```
# User session tokens
KEY: session:{session_id}
VALUE: {user_id, username, role, permissions}
TTL: 7 days (604800 seconds)

# User preferences cache
KEY: user:preferences:{user_id}
VALUE: JSON preferences object
TTL: 24 hours (86400 seconds)
```

9.1.2 Content Cache

```
# Recently trending articles
KEY: trending:articles:daily
VALUE: List of article IDs
TTL: 6 hours (21600 seconds)

# Repository trending data
KEY: trending:repos:weekly
VALUE: List of repository IDs with scores
TTL: 24 hours (86400 seconds)

# Technology trends
KEY: trends:tech:{date}
VALUE: Technology trend scores and mentions
TTL: 48 hours (172800 seconds)
```

9.1.3 Aggregate Caches

```
# User bookmark count
KEY: user:bookmarks:count:{user_id}
VALUE: Integer count
TTL: 1 hour (3600 seconds)

# Collection metadata
KEY: collection:{collection_id}
VALUE: Collection object with item count
TTL: 2 hours (7200 seconds)

# Search results cache
KEY: search:results:{query_hash}
VALUE: Paginated search results
TTL: 30 minutes (1800 seconds)
```

9.1.4 Task Queue Cache

```
# Workflow execution tracking
KEY: workflow:run:{run_id}
VALUE: Execution status and progress
TTL: Until completion + 7 days

# Agent task progress
KEY: agent:task:{task_id}:progress
VALUE: Percentage completed, current step
TTL: Until completion + 24 hours

# Celery task results
KEY: celery:result:{task_id}
VALUE: Task result object
TTL: 1 day (86400 seconds)
```

9.2 Cache Invalidation Strategy

1. **Event-Driven Invalidation**: Invalidate caches on INSERT/UPDATE/DELETE
2. **TTL-Based Expiration**: Natural expiration for time-sensitive data
3. **Bulk Invalidation**: Clear category caches after scraping operations
4. **User-Specific**: Invalidate on profile or preference changes
5. **Pattern Matching**: Use Redis SCAN for wildcard invalidation

Chapter 10

Entity Relationship Overview

10.1 Core Relationships

- **users** ← One-to-Many → user_bookmarks
- **users** ← One-to-Many → collections
- **users** ← One-to-Many → automation_workflows
- **users** ← One-to-Many → agent_tasks
- **users** ← One-to-Many → generated_documents
- **users** ← One-to-Many → notifications
- **users** ← One-to-Many → user_activity
- **sources** ← One-to-Many → articles
- **collections** ← One-to-Many → collection_items
- **automation_workflows** ← One-to-Many → workflow_runs

10.2 Polymorphic Relationships

Several tables use polymorphic relationships via `content_type` + `content_id`:

- **user_bookmarks**: References articles, repositories, research_papers, videos
- **collection_items**: References articles, repositories, research_papers, videos

These relationships are maintained by application logic rather than database constraints for flexibility.

Chapter 11

Database Performance Considerations

11.1 Partitioning Strategy

For production deployments with large data volumes:

11.1.1 Time-Based Partitioning

```
-- Partition articles by month
CREATE TABLE articles_2024_01 PARTITION OF articles
    FOR VALUES FROM ('2024-01-01')
    TO ('2024-02-01');

CREATE TABLE articles_2024_02 PARTITION OF articles
    FOR VALUES FROM ('2024-02-01')
    TO ('2024-03-01');
```

11.1.2 Range-Based Partitioning

```
-- Partition user_activity by user_id ranges
CREATE TABLE user_activity_0_250k PARTITION OF user_activity
    FOR VALUES FROM (0) TO (250000);

CREATE TABLE user_activity_250k_500k PARTITION OF user_activity
    FOR VALUES FROM (250000) TO (500000);
```

11.2 Query Optimization Tips

1. **Use EXPLAIN ANALYZE**: Profile slow queries before optimization
2. **Avoid SELECT ***: Specify only required columns
3. **Batch Operations**: Use INSERT INTO ... SELECT for bulk operations
4. **Connection Pooling**: Implement PgBouncer for connection management

5. **Read Replicas**: Direct read-heavy queries to replicas
6. **Prepared Statements**: Prevent SQL injection and improve planning
7. **Vacuum Strategy**: Regular VACUUM ANALYZE on high-churn tables
8. **Statistics**: Keep table statistics current with ANALYZE

11.3 Monitoring and Maintenance

11.3.1 Key Metrics to Monitor

```
-- Monitor table sizes
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename))
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;

-- Monitor index usage
SELECT schemaname, tablename, indexname, idx_scan, idx_tup_read
FROM pg_stat_user_indexes
ORDER BY idx_scan DESC;

-- Monitor slow queries
SELECT query, calls, total_time, mean_time
FROM pg_stat_statements
WHERE query NOT LIKE '%pg_stat%'
ORDER BY mean_time DESC
LIMIT 10;
```

11.3.2 Maintenance Operations

- **VACUUM**: Run daily on high-churn tables (articles, user_activity)
- **ANALYZE**: Update statistics weekly or after bulk operations
- **REINDEX**: Rebuild fragmented indexes monthly
- **AUTOVACUUM**: Configure aggressive settings for busy tables
- **Log Archival**: Manage transaction logs with WAL archiving

Chapter 12

Data Migration and Backup Strategy

12.1 Backup Approach

- **Full Backups**: Daily pg_dump to cloud storage
- **WAL Archiving**: Continuous WAL archiving for point-in-time recovery
- **Replication**: Streaming replication to standby server
- **Retention**: Keep 30 days of backups minimum

12.2 Migration Path

For schema updates in production:

1. Create migration scripts using Alembic or Flyway
2. Test migrations on staging environment
3. Schedule during low-traffic windows
4. Verify data integrity post-migration
5. Maintain rollback capability

Chapter 13

Security and Compliance

13.1 Access Control

```
-- Create read-only role for reporting
CREATE ROLE readonly;
GRANT CONNECT ON DATABASE synapse TO readonly;
GRANT USAGE ON SCHEMA public TO readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;

-- Create application role with limited permissions
CREATE ROLE app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE ON articles TO app_user;
GRANT SELECT, INSERT, UPDATE ON user_bookmarks TO app_user;
```

13.2 Sensitive Data Handling

- **Password Hashes**: Use Bcrypt/Argon2, never store plaintext
- **API Keys**: Encrypt and store in separate secrets management system
- **PII**: Consider encryption-at-rest for sensitive user data
- **Audit Logging**: Track who accesses sensitive data

Chapter 14

Conclusion

The SYNAPSE database schema provides a robust, scalable foundation for a FAANG-style technology intelligence platform. Key features include:

1. **Comprehensive Coverage**: All content types and user interactions
2. **Performance Optimized**: Strategic indexing across all access patterns
3. **Flexible Architecture**: JSONB and array columns for extensibility
4. **Scalability Ready**: Partitioning, replication, and caching strategies
5. **Security Conscious**: Role-based access control and data protection
6. **Operational Maturity**: Monitoring, backup, and maintenance guidelines

This schema supports real-time content aggregation, AI-powered analysis, user-driven workflows, and advanced analytics while maintaining data integrity and performance across millions of records.